

خلاصه امتحانی فصل ۷

مدل‌های اسپایکی مدرن

مرور تفکیک‌شده مفاهیم، معماری‌ها و فرمول‌ها

بخش اول: Spiking Transformer و SpikeFormer

۱.۱ از Deep SNN به Transformer اسپایکی

- در SNN، داده زمان‌محور و ورودی و خروجی معمولاً اسپایک‌های صفر و یک‌اند؛ مزیت اصلی، پردازش رویدادمحور و مصرف انرژی کمتر است.
- در Deep SNN نظارت‌شده، عبور گرادیان از تابع گسسته اسپایک با گرادیان جایگزین (Surrogate Gradient) ممکن می‌شود.
- لایه خطی اسپایکی، وزن را در اسپایک ضرب و سپس خروجی اسپایکی جدیدی تولید می‌کند:

$$W \times \text{spike}$$

- **SpikeFormer**: ترکیب معماری Transformer با شبکه عصبی اسپایکی برای دستیابی به مدل قدرتمندتر و کم‌مصرف‌تر.
- سه جزء ضروری:
 ۱. کدگذاری داده ورودی به اسپایک؛
 ۲. توجه اسپایکی؛
 ۳. بلوک MLP اسپایکی.

۲.۱ کدگذاری ورودی

- داده پیوسته باید به دنباله‌ای از صفر و یک در چند گام زمانی تبدیل شود.
- **Time Step** با T نشان داده می‌شود؛ مثلاً $T = 4$ یعنی هر مقدار در چهار گام زمانی بازنمایی می‌شود.
- دو هدف کدگذاری:
 - تبدیل مقدار پیوسته به اسپایک؛
 - پخش اطلاعات در طول زمان، به طوری که مقدار بزرگ‌تر معمولاً نرخ اسپایک بیشتری بسازد.

۳.۱ کدگذاری تصویر و Rate Encoding

نرمال‌سازی پیکسل

$$x_{\text{norm}} = \frac{x}{255}$$

- برای $T = 4$ ، تعداد تقریبی اسپایک از $x_{\text{norm}}T$ به دست می‌آید.
- مثال:

$$[0.2, 0.8, 0.5, 0.5] \times 4 = [0.8, 3.2, 2, 2]$$

$$\text{round}(\cdot) = [1, 3, 2, 2]$$

یعنی ویژگی‌ها به ترتیب یک، سه، دو و دو اسپایک در کل بازه تولید می‌کنند.

- **Rate Encoding**: مقدار عددی با تعداد یا نرخ اسپایک نمایش داده می‌شود:

$$x = 0.8, \quad T = 4 \quad \Rightarrow \quad xT \approx 3 \quad \Rightarrow \quad [1, 1, 1, 0]$$

ترتیب دقیق می‌تواند متفاوت باشد؛ تعداد اسپایک‌ها معنای اصلی را حمل می‌کند.

۴.۱ کدگذاری مبتنی بر Integration

- مقدار پیوسته در هر گام به پتانسیل اضافه می‌شود؛ با عبور از آستانه، اسپایک تولید و آستانه از پتانسیل کم می‌شود.
- برای $x = 0.8$ و $\theta = 1$:
- ۱. $0.8 < 1$: بدون اسپایک.
- ۲. $0.8 + 0.8 = 1.6$: اسپایک؛ باقی مانده 0.6 .
- ۳. $0.6 + 0.8 = 1.4$: اسپایک؛ باقی مانده 0.4 .
- ۴. $0.4 + 0.8 = 1.2$: اسپایک.

$$[0, 1, 1, 1]$$

- حداقل شرط کدگذاری معنادار: رابطه مستقیم مقدار پیوسته با نرخ اسپایک.

مبادله اصلی

$$\text{Energy/Cost} \leftrightarrow \text{Accuracy}$$

اسپایکی کردن داده می‌تواند بخشی از اطلاعات پیوسته را از بین ببرد؛ بنابراین SNN معمولاً انرژی کمتری مصرف می‌کند، اما ممکن است دقت پایین‌تری از Transformer معمولی داشته باشد.

۵.۱ هدف Attention اسپایکی

- مانند توجه معمولی، بخش‌های مرتبط با توجه به زمینه به یکدیگر وزن بیشتری می‌دهند.
- در متن، زمینه «جنگل» باید بازنمایی «شیر» را به معنای حیوانی نزدیک کند؛ در تصویر نیز ناحیه‌های مرتبط برجسته می‌شوند.
- در فضای اسپایکی، ارتباط بیشتر معمولاً با نرخ یا هم‌وقوعی بیشتر اسپایک‌ها نشان داده می‌شود.

۶.۱ Key، Query و Value اسپایکی

- سه ماتریس وزن جداگانه داریم:

$$W_Q, \quad W_K, \quad W_V$$

- دنباله اسپایکی ورودی در این وزن‌ها ضرب می‌شود و Q ، K و V اسپایکی ساخته می‌شوند.
- وزن‌ها در آموزش با Surrogate Gradient به‌روزرسانی می‌شوند.

۷.۱ Attention معمولی در برابر Spiking Attention

- توجه کلاسیک:

$$\text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V$$

Softmax نمره‌ها را به وزن‌هایی نرمال می‌کند که مجموعشان برابر یک است.

- توجه اسپایکی:

$$QK^TV$$

ضرب اسپایک‌های دودویی خود نوعی هم‌فعالی یا شباهت می‌سازد؛ بنابراین Softmax ضرورت و معنای قبلی را ندارد. مقیاس‌گذاری با $\sqrt{d}$ همچنان می‌تواند باقی بماند.

- حذف Softmax اجازه می‌دهد ترتیب ضرب‌ها تغییر کند:

$$(QK^T)V = Q(K^TV)$$

ابتدا محاسبه K^TV و سپس ضرب در Q می‌تواند پیچیدگی، حافظه و انرژی را کاهش دهد.

۸.۱ معماری کلی SpikeFormer

۱. کدگذاری ورودی پیوسته به دنباله اسپایکی.
۲. ورود به توجه خودی اسپایکی (Spiking Self-Attention; SSA).
۳. عبور خروجی توجه از بلوک MLP اسپایکی.
۴. استفاده از اتصال‌های Residual اطراف بلوک توجه و MLP.
۵. تصمیم نهایی با سر طبقه‌بندی یا خروجی وظیفه.

۹.۱ نقش LIF در SpikeFormer

- پس از بسیاری از لایه‌های خطی، نورون LIF قرار می‌گیرد.
- وظایف LIF: تجمع زمانی پتانسیل، نشت، مقایسه با آستانه، تولید اسپایک، ریست و دوره تحرک ناپذیری.
- MLP معمولی:

Linear → Activation → Linear

- MLP اسپایکی:

Linear → LIF → Linear → LIF

- LIF معادل دقیق تابع فعال‌ساز نیست، اما غیرخطی بودن ایجاد و خروجی را دوباره اسپایکی می‌کند.

۱۰.۱ Residual Connection

- اطلاعات قبلی را از مسیر میان‌بر حفظ می‌کند.
- به کاهش مشکل vanishing gradient کمک می‌کند.
- اگر یک بلوک بازنمایی مفیدی نسازد، بازنمایی قبلی کاملاً از بین نمی‌رود.

بخش دوم: SpikeBERT و تقطیر دانش

۱.۲ چرا متن برای SNN دشوارتر است؟

- تراکم اطلاعات (Information Density): تغییر کوچک مانند افزودن «ن» می‌تواند معنای جمله را معکوس کند؛ تصویر معمولاً افزونگی بیشتری دارد.
- وابستگی بلندمدت (Long-Term Dependency): معنای کلمه ممکن است به بخش‌های بسیار دور متن وابسته باشد، اما نشت LIF نوعی فراموشی ایجاد می‌کند.
- ماهیت نمادین زبان (Symbolic Nature): توکن گسسته ابتدا به embedding پیوسته و سپس دوباره به اسپایک تبدیل می‌شود؛ احتمال از دست رفتن ظرافت معنایی افزایش می‌یابد.
- SpikeFormer و SpikeBERT بیشتر اثبات مفهوم (Proof of Concept) هستند؛ امکان تبدیل معماری را نشان می‌دهند، اما هنوز چالش دقت و همگرایی دارند.

۲.۲ Knowledge Distillation

- آموزش SpikeBERT فقط با برچسب نهایی به خوبی همگرا نمی‌شود؛ بنابراین خروجی‌های میانی نیز راهنمایی می‌شوند.
- Teacher**: مدل BERT معمولی، از پیش آموزش دیده و دارای بازنمایی‌های پیوسته زبانی.
- Student**: مدل اسپایکی SpikeBERT.
- دانش از چهار مسیر منتقل می‌شود:
 - Embedding Loss
 - Hidden Feature Loss
 - Logit Loss
 - Cross Entropy Loss نسبت به برچسب واقعی

۳.۲ تبدیل Embedding به اسپایک

- شکل ورودی پیوسته:

$$X \in \mathbb{R}^{N \times D}$$

N : تعداد توکن‌ها؛ D : تعداد ویژگی‌های هر توکن.

- با افزودن T گام زمانی:

$$N \times D \longrightarrow T \times N \times D$$

- مثال با $T = 5$ ، مقدار 0.3 و آستانه ۱:

اسپایک و ریست $0.3 \rightarrow 0.6 \rightarrow 0.9 \rightarrow 1.2 \rightarrow$

۴.۲ بازگردانی و هم‌ترازسازی بازنمایی

- دریافت embedding اولیه از BERT.
- تبدیل مقدار پیوسته به اسپایک.
- پردازش در SpikeBERT.
- تبدیل تعداد یا نرخ اسپایک به مقدار پیوسته.
- نگاشت خروجی Student به فضای Teacher.
- مقایسه و محاسبه خطا.

Projection

$$H' = HW + b$$

یا

$$H' = \text{MLP}(H)$$

MLP نسبت به یک لایه خطی ساده، نگاشت قوی‌تری برای هم‌ترازکردن فضای دو مدل می‌سازد.

۵.۲ Embedding Loss و Feature Loss

- خروجی‌های متناظر Teacher و Student پس از projection خانه‌به‌خانه مقایسه می‌شوند.
- مثال:

$$H_T = [1, 2, 0], \quad H_S = [0.5, 0.3, 1]$$

$$L = (1 - 0.5)^2 + (2 - 0.3)^2 + (0 - 1)^2$$

- لازم نیست همه لایه‌ها هم‌تراز شوند؛ انتخاب لایه‌های مقایسه‌شونده بخشی از طراحی مدل است.

۶.۲ دو مرحله آموزش SpikeBERT

- مرحله اول - انتقال دانش عمومی:
 - نزدیک کردن embedding ها و ویژگی‌های میانی Student به Teacher.
 - تمرکز اصلی روی Embedding Loss و Hidden Feature Loss.
- مرحله دوم - آموزش مخصوص وظیفه:
 - تنظیم مدل برای وظیفه‌ای مانند تحلیل احساسات.
 - مقایسه logit های دو مدل و خروجی Student با برچسب واقعی.

۷.۲ Cross Entropy و Logit Loss

- مثال خطای لاجیت:

$$z_T = [0.2, 0.1], \quad z_S = [0.5, 0]$$

$$L_{\text{logit}} = (0.2 - 0.5)^2 + (0.1 - 0)^2$$

- خطای آنتروپی متقاطع برای کلاس درست y :

$$L_{\text{CE}} = -\log \left(\frac{e^{z_y}}{\sum_j e^{z_j}} \right)$$

z_y : لاجیت کلاس صحیح؛ مخرج: مجموع نمایی لاجیت همه کلاس‌ها.

۸.۲ تابع زیان نهایی و آموزش

زیان ترکیبی SpikeBERT

$$L_{\text{total}} = \lambda_1 L_{\text{embedding}} + \lambda_2 L_{\text{feature}} + \lambda_3 L_{\text{logit}} + \lambda_4 L_{\text{CE}}$$

- λ_i : وزن اهمیت هر مؤلفه خطا.
- پس انتشار بر اساس L_{total} انجام و برای عبور گرادینان از اسپایک‌ها از Surrogate Gradient استفاده می‌شود.
- حتی embedding اولیه SpikeBERT نیز در آموزش fine-tune می‌شود.

۹.۲ نقش Time Step

- T یک ابرپارامتر ویژه شبکه اسپایکی و تعداد گام‌های زمانی کدگذاری است:

$$T = \text{Time Step}$$

- افزایش T مانند تجمع شواهد (Evidence Accumulation) است؛ شواهد بیشتری برای عبور از آستانه و تصمیم‌گیری فراهم می‌شود.
- رابطه دقت تا نقطه‌ای صعودی و سپس اشباعی است:

$$T \uparrow \implies \text{Accuracy} \uparrow \quad \text{تا نقطه اشباع}$$

- هدف: یافتن کمترین T که بهترین دقت عملی یا temporal resolution مناسب را ایجاد کند.
- مشکلات T بسیار بزرگ:
 - پایان اطلاعات مفید و حساسیت بیشتر به نویز؛

- افزایش بار حافظه و ارتباط با محدودیت حافظه فعال؛
- کندتر شدن همگرایی، آموزش و inference؛
- هزینه بیشتر در برابر بهبود اندک، ثابت ماندن یا حتی افت دقت.

۱۰.۲ مثال Attention اسپایکی: شیر و جنگل

- با فرض $Q = K = V = X$ ، امتیاز «شیر به شیر» برابر ۴ و «شیر به جنگل» برابر ۲ است.

$$۴ + ۲ = ۶, \quad A_{\text{شیر, شیر}} = \frac{۴}{۶} = \frac{۲}{۳}, \quad A_{\text{شیر, جنگل}} = \frac{۲}{۶} = \frac{۱}{۳}$$
- بازنمایی جدید:

$$V'_{\text{شیر}} = \frac{۲}{۳}V_{\text{شیر}} + \frac{۱}{۳}V_{\text{جنگل}}$$

زمینه «جنگل» باید بعد حیوانی «شیر» را تقویت و بعد مایع را تضعیف کند.

جمع‌بندی مقایسه‌ای

- **SpikeFormer**: تمرکز بر معماری Transformer اسپایکی، کدگذاری، توجه بدون Softmax و MLP مبتنی بر LIF.
- **SpikeBERT**: انتقال همین ایده به زبان با کمک Teacher-Student و چند زیان میانی و نهایی.
- **مبادله مشترک**: کاهش انرژی و پیچیدگی در برابر خطر افت دقت و ازدست‌رفتن اطلاعات پیوسته.

بخش سوم: SpikeGPT، RWKV و تولید متن اسپایکی

۱.۳ طبقه‌بندی در برابر تولید

- **طبقه‌بندی (Classification)**:
 - BERT دوطرفه و encoder-only است؛ برای تصمیم نهایی کل متن را از دو جهت می‌بیند.
 - hidden state می‌توانند average pooling شوند و سپس به سر طبقه‌بندی برسند.
- **تولید (Generation)**:
 - GPT علی، خودرگرسیو و decoder-only است.
 - با masked self-attention فقط گذشته را می‌بیند و توکن بعدی را پیش‌بینی می‌کند.
 - در آموزش، هدف‌ها یک موقعیت شیف‌ت می‌یابند:

$$[x_1, x_2, \dots, x_{s-1}] \longrightarrow [x_2, x_3, \dots, x_s]$$

- خروجی به واژگان متصل و معمولاً با Cross Entropy آموزش داده می‌شود.

۲.۳ چرا SpikeGPT از RWKV استفاده می‌کند؟

- تبدیل مستقیم Transformer به مدل اسپایکی برای تولید متن معمولاً کافی و کارآمد نیست.
- توجه کامل همه توکن‌ها را دوباره دو مقایسه می‌کند و پیچیدگی تقریبی دارد:

$$O(n^2)$$

- RNN گذشته را در یک state خلاصه می‌کند، اما غیرخطی بودن‌هایی مانند tanh روی حالت، فراموشی و vanishing gradient ایجاد می‌کنند.
- **RWKV**: ترکیب حالت بازگشتی RNN با مزایای Transformer؛ حالت خطی‌تر است و گذشته بدون توجه دوباره دوی کامل خلاصه می‌شود.

۳.۳ چهار جزء RWKV

- **Receptance (R):** گیت خروجی؛ تعیین می‌کند چه مقدار از state عبور کند.
- **Weight (W):** عامل فراموشی؛ مقدار حفظ یا کاهش اطلاعات قبلی.
- **Key (K):** اهمیت توکن فعلی.
- **Value (V):** محتوای توکن فعلی.
- در RWKV، Query حذف می‌شود؛ گذشته در یک state جمع شده است.

RWKV خروجی

$$\text{Output} = R \times \text{State}$$

۴.۳ مثال عددی به‌روزرسانی State

۱. توکن اول:

$$K_1 = 2, \quad V_1 = 10, \quad \text{State}_1 = \frac{K_1 V_1}{K_1} = \frac{20}{2} = 10$$

۲. توکن دوم و عامل فراموشی:

$$K_2 = 3, \quad V_2 = 4, \quad R_2 = 0.9, \quad W = 0.5$$

۳. صورت و منخرج جدید:

$$0.5(20) + 3(4) = 22, \quad 0.5(2) + 3 = 4$$

۴. حالت و خروجی:

$$\text{State}_2 = \frac{22}{4} = 5.5$$

$$\text{Output} = 0.9(5.5) = 4.95$$

دو معنای متفاوت زمان

- t در RWKV: موقعیت توکن در دنبالهٔ زبانی.
- T در SNN: تعداد گام‌های زمانی برای کدگذاری اسپایکی هر نمایش.
- SpikeGPT هم‌زمان با زمان ترتیبی زبان و زمان اسپایکی سروکار دارد.

۵.۳ روند معماری SpikeGPT

۱. تبدیل embedding ورودی به نمایش باینری یا اسپایکی با LIF.
۲. ورود دادهٔ اسپایکی به بلوک RWKV.
۳. اسپایکی کردن دوبارهٔ خروجی RWKV.
۴. ورود به بخش فیدفوروارد اصلاح شده یا R-FFN.
۵. اسپایکی کردن دوبارهٔ خروجی R-FFN.
۶. استفاده از خروجی برای تولید متن یا طبقه‌بندی.

۶.۳ Channel-Mix و Time-Mix

- **Time-Mix (RWKV):** اطلاعات توکن‌های قبلی و توکن فعلی را ترکیب می‌کند؛ معادل نقش فهم زمینه در Attention.
- **Channel-Mix (R-FFN):** روی ویژگی‌ها و کانال‌های توکن فعلی کار می‌کند؛ معادل نقش MLP یا feed-forward.

- در R-FFN، بعد نمایش ابتدا بزرگ، سپس از ReLU عبور و دوباره فشرده می‌شود؛ مسیر گیت با محتوای اصلی عنصر به عنصر ترکیب می‌شود.

ضرب هادامارد در R-FFN

$$[a_1, a_2, a_3] \odot [b_1, b_2, b_3] = [a_1 b_1, a_2 b_2, a_3 b_3]$$

۷.۳ پیوستگی میانی، تنگی و مصرف انرژی

- لازم نیست همه نقاط SNN صفر و یک باشند؛ مقدارهای پیوسته میانی عمدتاً پتانسیل غشا هستند.
- خروجی لایه‌های اسپایکی بسیار تنگ (Sparse) است؛ فقط رخدادهای فعال محاسبه ایجاد می‌کنند.
- GPU برای عملیات متراکم طراحی شده است؛ مزیت واقعی SNN روی سخت‌افزار نورومورفیک آشکارتر می‌شود.
- عملیات شبکه غیر اسپایکی:

$$\text{MAC} = \text{Multiply} + \text{Accumulate}$$

- با ورودی صفر و یک در شبکه اسپایکی، بسیاری از ضرب‌ها حذف می‌شوند:

$$\text{AC} = \text{Accumulate}$$

- برآورد مقاله SpikeGPT: کاهش انرژی تقریبی ۳۲ برابری نسبت به Vanilla GPT.

جمع‌بندی فصل

- **SpikeFormer**: توجه اسپایکی کم‌هزینه‌تر برای معماری‌های عمیق.
- **SpikeBERT**: حل دشواری متن با تقطیر دانش از BERT.
- **SpikeGPT**: تولید خودرگرسیو با RWKV، Time-Mix و R-FFN به جای توجه کامل.